

附录 | 术语表

术语表

术语	解释
Data + AI	用数据工程、系统工程和运行保障把 AI 能力真正接到业务系统上。
RAG	Retrieval-Augmented Generation, 先检索再生成的知识服务链路。
Agent	不只回答问题，还会调用工具、执行动作、管理流程状态的系统。
Source / Raw Resource	业务世界里的原始输入对象，例如 ticket、文档、音频、视频、数据库变更流。
Input Asset	已经过准入、可进入工程链路的输入资产。
Serving Object	最终供检索、引用、分析或生成消费的对象。
数据契约 (Data Contract)	系统可消费的输入准入规则，至少应覆盖 shape、语义、证据、策略和质量边界。
Source Manifest	一次运行的输入声明，说明这次 ingest 准备读什么、从哪读、如何读。
Gate	进入运行链路前的校验和决策机制，不只是 pass / fail，还可能 quarantine、warn 或 reject。
Metadata	帮系统理解和消费数据的最小上下文信息。
PII	Personally Identifiable Information，能识别或重建个人身份的信息。
Batch Ingest	按批次边界做输入接入和落库的链路。
Incremental Ingest	基于游标、时间戳、LSN 或变更序列持续追加读取的链路。
CDC	Change Data Capture，从源系统的变更流中捕获插入、更新、删除事件。
Cursor	下次应该从哪里继续读取。
Watermark	当前批次已经承认到哪里。
Checkpoint	持久化保存的状态边界，让链路能恢复、回放和继续执行。
Dedupe Key	在输入层判断是不是同一业务事件的键。
Idempotency Key	在写入层防止重复 side effect 的键。
Retry	同一次执行里对瞬时失败进行重试。

术语	解释
Rerun	重新跑同一个作业定义，通常不改变输入语义边界。
Replay	重放同一批或同一来源输入，重建当时那次接入。
Restore	先回到已知可用状态，再决定后续是否 replay / backfill。
Backfill	补历史空洞、补旧分区或补错过窗口。
Runbook	面向团队交接和故障恢复的可执行操作手册。
Provenance	结果来源、过程和责任链信息。
Lakehouse	同时兼顾数据湖灵活性和表状态管理能力的数据底座。
Apache Iceberg	本课程 Week04 使用的开放表格式, 用 metadata、snapshot、manifest 和 data file 管理表状态。
Pylceberg	Python 侧操作 Iceberg catalog、table、snapshot 和 metadata 的核心依赖。
Snapshot	某个时刻一张表被提交后的稳定状态。
Manifest List	指向一组 manifest 的索引层。
Iceberg Manifest	描述某次提交涉及哪些 data file / delete file 的元数据对象。
Metadata Log	记录表状态演进历史的元数据链。
Metadata Pointer	指向当前 table metadata 文件的入口，决定读者看到哪一个表状态。
Time Travel	回到某个旧 snapshot 所代表的状态集合。
Schema Evolution	在不破坏状态链的前提下演进表结构。
Hidden Partitioning	将分区逻辑交给表格式管理，不要求业务层直接暴露分区键。
Catalog	管理 namespace、table metadata 和表注册入口的组件。
SQL Catalog	Week04 本地实现里基于 PostgreSQL 的 Iceberg catalog，负责登记 namespace、table 与 meta-data 位置。
Warehouse	表数据和元数据默认落盘的根路径。
Table Location	某张表在 warehouse / object storage 中的实际落点，不能和 catalog 概念混在一起。
Data File	Iceberg 表实际承载数据的文件，Week04 当前以 Parquet 文件作为核心对象。

术语	解释
Bronze / Silver	Week04 当前主线中的最小两层表设计：Bronze 保留更接近原始输入的状态，Silver 提供更稳定的消费视图。
Baseline	当前系统在对象、状态、性能和运行习惯上的最小可验收基线。
Materialization Report	Week04 记录 Pylceberg materialize 结果、表名、行数、文件和执行状态的 JSON 证据。
Course Site Sync Packet	OmniSupport Copilot 项目内用于同步课程站点的 Week04 实现说明，防止讲义命令和真实仓库脱节。
Devbox CLI	Week04 学生执行 Pylceberg 命令的 Docker devbox 入口，当前是比 Dagster wrapper 更直接的验证路径。
Transform	将稳定数据状态加工成业务可消费数据产品的过程，本课程 Week05 主要通过 dbt Core 表达。
dbt sources	dbt 中对真实上游表的声明，负责把 source name、schema、table、字段和 owner 写清楚。
Staging Model	dbt 分层中的输入规范层，负责字段重命名、类型转换和最小清洗。
Intermediate Model	承接业务组合逻辑的中间层，避免把复杂 join 或派生逻辑塞进 staging 或 mart。
Mart	面向下游消费的数据产品层，例如 <code>support_case_mart</code> 和 <code>support_kpi_mart</code> 。
Metric Registry	指标注册表，记录 metric name、model、measure、维度白名单、过滤器白名单和 owner。
Semantic Layer	让不同消费方复用同一套指标名、维度、过滤器和口径边界的语义接口层。
MetricFlow	dbt 生态中用于表达和查询语义指标的能力，本课程 Week05 将其作为扩展方向而非学生核心依赖。
Tool Contract	Agent 工具可调用接口的输入输出、权限、审计和负例边界定义。
Controlled Metric Query	受控指标查询路径，Agent 只能通过 schema、registry、白名单和审计约束后的接口查询指标。
Dagster Asset	用代码定义的数据资产，包含 asset key、上游依赖、计算/观察逻辑和 metadata。
Asset Graph	以资产而不是脚本为中心的依赖图，表达哪些数据产品依赖哪些上游状态。

术语	解释
Definitions	Dagster 项目入口，用来注册 assets、jobs、resources、checks 和 schedules。
Materialization	对某个资产执行计算或写入，使它在当前分区/状态下可消费。
Asset Observation	对外部或 source asset 的状态观察，不等同于 materialization。
External Asset	由外部系统负责生成，但在当前 asset graph 中需要被依赖或观测的资产。
Partition	资产可独立运行、回填和验证的边界，Week06 Student Core 默认使用 daily partition。
Asset Check	针对某个资产当前状态的质量判断，例如 row count、duplicate、required field null rate。
Run Evidence	描述一次资产运行事实、质量结果、分区、reason code 和下游决策的证据文件。
Data Factory Runbook	Week06 的可交接操作手册，指导观察、物化、补数、检查、证据记录和下游放行。
Parsed Document	从 raw document 解析出的结构化文档表示，保留 layout、hierarchy、tables、page、bbox 和 provenance。
DoclingDocument	Docling 的统一文档表示，可表达文本、表格、图片占位、标题层级、页码、bbox 与 provenance。
Knowledge Section	Week7 解析后的文档结构单元，通常继承 <code>section_path</code> 、 <code>page_no</code> 、 <code>bbox</code> 和 source metadata。
Document Chunk	Week8 索引前的候选文本单元，由结构感知切片生成，不等同于原始 section。
Evidence Anchor	让 chunk 回指原始文档位置的证据对象，是 citation、bad case replay 和质量复核的事实来源。
Section-aware Chunking	先尊重标题层级、表格、页码和 section 边界，再处理 token budget 的切片策略。
BBox	Bounding Box, 元素在页面中的坐标范围，常用于 PDF citation 和位置回指。
Source Fingerprint	对原始文档字节计算的稳定指纹，用于确认解析对象和 ingest 对象一致。
Parse Run	一次文档解析运行记录，保存 parser、策略版本、输入输出、状态和错误。
Chunk Quality Sample	对 chunk 和 evidence anchor 的抽样质检记录，用于 Week8 准入和后续回归对比。

术语	解释
Week8 Ready Gate	Week7 输出给 Week8 的索引准入结论, 说明哪些 chunks 可消费、哪些必须 blocked。

Week07 | 非结构化文档资产补充术语

术语	小白解释	工程解释 / OmniSup-port 例子	常见误解
Raw Document Asset	原始文档文件, 不是解析后的文本	Week3 接入的 PDF / HTML / Markdown / DOCX, 必须有 raw object path 和 source fingerprint	以为本地一个 PDF 文件名就能代表资产
Parsed Document	parser 读懂后的文档结构	DoclingDocument 或 normalized representation, 保留页面、标题、表格和 provenance	以为 parsed document 就是一串 text
Parser Adapter	把不同 parser 输出归一成课程对象的适配层	Docling-first, Unstructured optional fallback, OCR disabled by default	做工具横评而不写 schema
Parser Capability Report	解析能力报告	记录 backend、OCR、fallback、sections、tables、warnings	只记录成功, 不记录缺失能力
Provenance	这段内容从哪里来、怎么来的	page、bbox、span、source fingerprint、parser strategy	只写 URL 就算 provenance
Knowledge Section	文档结构单元	<code>section_path=["Setup", "SSO"]</code> 直接把 section 当最终带 page / bbox / section_type 检索 chunk	
Section Path	标题层级路径	Help Center 的 Setup > SSO > Troubleshooting	丢标题后让 Week8 去猜上下文
Section-aware Chunking	先尊重结构再控制长度	<code>section_aware_v1</code> 保持步骤、表格、FAQ 边界	固定字符数就是 chunking
Chunk Strategy Version	切片策略版本	<code>section_aware_v1</code> 写入 chunks, 策略变化可回归	只要 chunk 内容一样就不用版本
Stable Chunk ID	可复现的 chunk 身份	由 doc、fingerprint、strategy、sec-	用随机 UUID 或自增 ID

术语	小白解释	工程解释 / OmniSup- port 例子	常见误解
		tion、content hash 派 生	
Evidence Anchor	chunk 的证据锚点	<code>anchor_id</code> 连接 chunk、 section、doc、page、 bbox 和 source	让 LLM 临时写 citation
BBox Missing Reason	坐标缺失原因	<code>not_applicable_non_paged</code> <code>ocr_disabled_in_student_core</code>	坐标缺失直接忽略
Quality Gate	Week8 准入门禁	metadata、 anchor、 page/bbox、 empty、 orphan、 PII heuristic	报告里只写“通过”
Sample Shortfall	样本不足说明	不足 50 份文档时写当 前样本数和扩展计划	样本少就不报告
Week8 Ready Gate	Week7 交给 Week8 的 消费规则	<code>allowed_for_indexing</code> 、 <code>blocked chunks</code> 、 <code>warnings</code> 、 <code>consump- tion rule</code>	以为是展示用 JSON

Week05 | Transform、语义层与受控指标查询补充

Analytics / dbt 术语

术语	小白解释	工程解释 / OmniSup- port 例子	常见误区
Analytics Engineering	把业务分析口径做成工 程资产	Week05 用 <code>analytics/</code> 、 dbt mod- els、 tests、 docs 和 re- ports 交付指标包	以为只是写 SQL 报表
dbt Core	本地运行 dbt project 的开源工具	在 <code>analytics/</code> 里执行 <code>dbt debug</code> 、 <code>dbt build</code> 、 <code>dbt docs generate</code>	以为 dbt 是数据库或 BI
Source	dbt 承认的真实上游表	<code>omni_postgres.ticket_fac</code> <code>customer_dim</code> 、 <code>ticket_comment_fact</code> 、 <code>knowledge_doc</code>	在 source 层写复杂业 务逻辑
Staging	把 source 变成稳定输 入形状	<code>stg_tickets</code> 统一 sta- tus、priority、时间和 PII 状态	在 staging 里写最终 KPI
Intermediate	承接跨表组合和复杂派 生	<code>int_support_cases</code> 汇 总 ticket、 customer、 comment 逻辑	直接暴露给 Agent

术语	小白解释	工程解释 / OmniSup- port 例子	常见误区
Mart	面向下游消费的数据产品层	<code>support_case_mart</code> 、 <code>support_kpi_mart</code>	把所有字段都塞进 mart
Grain	一行代表什么	<code>support_kpi_mart</code> 是 <code>metric + date + dimensions</code>	先写 SQL，后补粒度解释
Data Test	对模型结果或 source 假设的断言	<code>not null</code> 、 <code>accepted values</code> 、 <code>relationships</code>	以为测试只是装饰
Unit Test	用小样本验证 SQL 逻辑边界	<code>reopen</code> 、 <code>escalation</code> 、 <code>first response</code> 规则	用它替代数据质量测试
Lineage	依赖关系和影响面	<code>source</code> → <code>staging</code> → <code>marts</code> → <code>registry</code> → <code>tool</code>	只把它当成好看的图
Artifact	dbt 运行生成的机器可读证据	<code>manifest.json</code> 、 <code>catalog.json</code> 、 <code>run_results.json</code>	把 target 当主要交付物
<code>manifest.json</code>	dbt 项目依赖清单	支撑 lineage、impact notes、资源映射	以为它是日志文件
<code>catalog.json</code>	warehouse 列和类型信息	支撑 dbt docs 的真实字段展示	以为它能替代模型描述
<code>run_results.json</code>	节点运行状态和耗时	支撑 build evidence 和 失败复盘	以为它是完整 lineage

Metric / Semantic / Tool 术语

术语	小白解释	工程解释 / OmniSup- port 例子	常见误区
Metric	团队共同承认的指标接口	<code>p1_ticket_count</code> 有 <code>source</code> 、 <code>grain</code> 、 <code>owner</code> 、 <code>tests</code> 、 <code>roles</code> 、 <code>audit</code>	以为 metric 就是一条 SQL
Measure	可聚合的数值或表达式	<code>metric_value</code> + <code>aggregation</code>	把 measure 当完整业务指标
Dimension	分组或过滤的业务属性	<code>product_line</code> 、 <code>priority</code> 、 <code>org_id</code> 、 <code>category</code>	以为所有列都能当维度
Entity	可识别或连接业务对象的键	<code>ticket_id</code> 、 <code>org_id</code>	把任意字段当 entity
Semantic Model	业务实体、维度、度量和关系的模型	未来可从 <code>registry</code> 迁移到 <code>semantic models</code>	以为必须先买平台

术语	小白解释	工程解释 / OmniSup- port 例子	常见误区
Semantic Graph	语义模型之间的关系图	case、customer、metric、safe view 之间的查询路径	只把它当视觉图
Semantic Layer	让不同系统复用同一指标含义的接口层	Week05 先用本地 registry 落最小语义契约	以为等于 dbt Cloud
MetricFlow	dbt 生态中的语义查询能力	本周作为扩展理解，不是 Student Core 硬依赖	以为必须生产化
Metric Registry	本地指标注册表	analytics/metric_registry 固定指标、维度、过滤器、角色和窗口	把它写成自由说明文档
Tool Contract	Agent 工具的输入输出和拒绝边界	query_support_kpis_v1.js 只写正例，不写拒绝定义 schema、denial code、audit	
JSON Schema	描述 JSON 结构和约束的标准	required、enum、additionalProperties 约束工具输入	以为 schema 等于权限
Structured Outputs	让模型输出严格匹配结构	连接工具调用结果和可审计 JSON	以为它能替代 runtime guard
Metric Whitelist	允许查询的指标清单	registry 中登记的 6 个核心指标	让 Agent 自由发明指标
Dimension Whitelist	允许切片的维度清单	product_line 、 priority 、 org_id 、 category	开放 customer_email 等敏感维度
Time Window Guard	查询时间窗口上限	max_window_days: 31	让 Agent 查无限历史
Role Filter	基于角色的查询限制	support_ops 、 instructor 、 admin	只靠前端隐藏按钮
Parameterized Query	参数化 SQL 查询	runtime 生成确定性 SQL，不拼 raw input	以为它能替代所有安全控制
Audit Log	谁查了什么的证据	actor、metrics、filters、release、row_count	当作可选字段
Denial Code	程序可识别的拒绝原因	METRIC_DENIED 、 ROLE_DENIED 、 DIMENSION_DENIED	只返回自然语言错误

Week06 | 资产化数据工厂补充术语

这些术语补充上方短定义，重点服务 Week06 的课时、实验和作业。读它们时不要把 Dagster 当成唯一目标，真正目标是把 Week03—Week05 的脚本、表、指标和证据组织成可运营的数据资产系统。

术语	小白解释	工程解释 / OmniSupport 例子	常见误区
Data Factory	不是“数据工厂”这个比喻, 而是一套能运行、补数、检查、留证据、可交接的数据资产系统	Week06 把 manifest、partitioned ingest、checks、run evidence 和 runbook 组织成 Data Factory v1	以为只要有 Dagster UI 就叫数据工厂
Dagster Asset	用代码定义和维护的数据资产	week06/silver/ticket_factories.py 每个 helper 函数都是资产, 不是普通 Python 函数	把每个 helper 函数都拆成 asset
Asset Key	资产在图中的稳定身份	week06/ops/run_evidence_report 让团队能定位证据资产	用动作名如 run_backfill_now 当资产名
Asset Graph	数据产品依赖图	从 seed manifest 到 ticket fact、backfill plan、run evidence 的依赖关系	把它当成装饰流程图
Materialization	资产被生成或更新的事件	对某个 daily partition 运行 materialize, 产生状态、metadata 或 report	以为 Dagster run 绿了就代表下游可消费
Asset Observation	观察外部资产状态	Week04 lakehouse 或 Week05 KPI mart 未由 Week06 生成时, 只观察状态	把 observation 当作本周已经生成了外部资产
External Asset	当前图依赖但不负责生成的资产	Week04 stable table、Week05 semantic mart 对 Week06 来说可以是 optional external asset	外部资产没跑通就 fake passed
Definitions	Dagster 项目入口	pipelines/definitions.py 注册 assets、jobs、resources、checks	新建多个平行入口, 导致 UI 和测试不一致
Resource	运行时依赖	Postgres、MinIO、report directory、env config 都是资源边界	在 asset 里硬编码本机路径
Partition	可独立运行、补数和验证的边界	课堂默认 2026-04-17 daily partition	把日期范围、批次和业务 ID 混成一个概念
Backfill	对历史空洞或旧分区做受控重算	先生成 reports/week06/backfill/	一失败就全量重跑

术语	小白解释	工程解释 / OmniSup- port 例子	常见误区
		<code>*.json</code> dry-run plan, 再决定是否执行	
Replay	重放同一批输入	用同一 source / manifest 语义重建一次接入	把 replay 和随手 rerun 混在一起
Restore	先回到已知好状态	先恢复到可信状态, 再判断是否 replay / backfill	把 restore 当成万能回滚按钮
Asset Check	针对资产当前状态的质量判断	manifest consistency、row count、duplicate、required fields、partition completeness	以为 pytest 通过就不需要 asset checks
Run Evidence	结构化运行证据	记录 asset、partition、status、reason code、report path、downstream decision	把日志文件当成 evidence
Downstream Decision	下游是否可消费的结论	<code>dry_run_only</code> 、 <code>hold_downstream</code> 、 <code>manual_review_required</code> 、 <code>proceed_to_week07</code>	只写“成功/失败”, 不告诉下游怎么办
Reason Code	可复核的状态原因	<code>dry_run_no_db_write</code> 、 <code>partition_already_included</code> 、 <code>week05_analytics_not_available</code>	只写自然语言, 后续无法统计和复盘
Optional Dependency	可观察但不强制通过的依赖	Week04 / Week05 没启用时写 <code>not_available</code> 或 <code>skipped</code>	未启用也写 <code>passed</code>
Data Factory Runbook	面向团队交接的操作接口	<code>runbooks/week06-data-factory.md</code> 说明 UI / CLI、checks、evidence 和恢复决策	写成作者个人笔记, 别人无法复现

Week08 | 检索、生成与 RAG 服务契约补充术语

术语	小白解释	工程解释 / OmniSup- port 例子	常见误区
RAG	检索增强生成	Week8 中是 retrieval、evidence、generation、citation、audit 的闭环	以为只是把资料塞进 prompt
Index Release	一版可复现的索引资产	<code>index-week08-dev</code> 绑定数据版本、chunk 策略、embedding 模型和构建统计	只记录 embedding 模型, 不记录数据和策略

术语	小白解释	工程解释 / OmniSup-port 例子	常见误区
Index Manifest	索引版本说明书	contracts/release/index-manifest-schema 约束构建统计和质量门禁	写成自由文本, 不能被测试
pgvector	PostgreSQL 的向量检索扩展	Week8 Student Core 的向量检索路径	以为必须上外部向量数据库
PostgreSQL FTS	PostgreSQL 全文检索	用于型号、错误码、关键词和术语召回	以为向量检索能替代所有关键词场景
RRF	Reciprocal Rank Fusion	用排序位置融合 vector 和 FTS 结果	直接相加不同尺度的 score
Metadata Filter	检索前过滤条件	product_line 、 visibility_scope 、 index_release_id 等必须在生成前生效	生成后再过滤权限
Rerank	二阶段重排	Cross-Encoder 只对 Top-N candidates 精排, 失败时 fallback 到 RRF	把 rerank 当成必需依赖导致 API 挂掉
Retrieval Debug	检索分数解释	返回 vector / fts / rrf / rerank / final score	只返回 content, 无法排查
Citation	引用证据对象	从 retrieval meta-data 投影 evidence_id 、page、section、source	让 LLM 临时编引用
Evidence ID	证据稳定身份	连接 Week7 anchor 与 Week8 citation	用自然语言段落代替 ID
Prompt as Code	把 prompt 文件化和版本化	prompt_manifest.yml 记录 prompt_release_id 和模板文件	prompt 藏在运行时字符串里
Abstain Reason	结构化拒答原因	no_retrieval_results 、 low_evidence_score 等	把拒答当成模型失败
RAG Audit Log	RAG 请求审计记录	记录 question、filters、evidence ids、scores、answer、release ids	只保存自然语言答案

Week09 | Agent Skills 与 Skill Pack 补充术语

术语	小白解释	工程解释 / Omnisupport 例子	常见误区
Agent Skill	Agent 可发现和执行的一套 workflow 能力	一个 skill 至少要有 <code>SKILL.md</code> , 可选 <code>scripts/</code> 、 <code>references/</code> 、 <code>assets/</code> , 用于封装数据契约检查、回填 <code>runbook</code> 、RAG 契约检查等工程工艺	把 skill 当成一段更长的 prompt
Skill Pack	多个 skills 的可迁移交付包	Week09 要交付 <code>omnisupport-skill-pack-v0.1</code> , 包含 5 个最小 skills、manifest、验证报告和交付总结	只把几个 Markdown 文件放进目录就算完成
SKILL.md	Skill 的入口说明书	说明触发条件、何时使用、输入输出、执行步骤、验证命令和不做事项	把它写成教程长文, Agent 需要读完才知道怎么做
Progressive Disclosure	按需加载上下文	先读 <code>SKILL.md</code> , 只有命中任务时再读 <code>scripts</code> 、 <code>references</code> 或 <code>examples</code>	一次性把所有文档塞进上下文
repo-scoped skills	仓库内随项目版本管理的 skills	课程建议 canonical source 放在 <code>skills/</code> , 再复制到 <code>.agents/skills/</code> 供本地 Agent 发现	只在个人机器安装, 团队和 CI 不可复现
<code>.agents/skills</code>	本地 Agent 发现入口	Week09 用 copy 脚本从 <code>skills/</code> 同步, 不建议用 symlink	直接手工复制后不再校验版本
Skill Manifest	Skill Pack 的清单	记录 pack name、version、canonical source、discovery target、skills 列表和验证项	只写 README, 不提供机器可读入口
Dry-run Skill	只生成计划和检查, 不产生破坏性副作用	<code>ingest-backfill-runbook</code> 先生成 dry-run plan, 再由 runbook 决定是否执行	让 Agent 直接修数据
Reference Policy	Skill 内引用资料的加载规则	<code>references/</code> 只放稳定契约、路径说明、run-	把所有历史材料都放进去导致上下文污染

术语	小白解释	工程解释 / OmniSup- port 例子	常见误区
		book 片段, 不塞整周讲 义	
Validation Report	Skill Pack 是否可发现、 可执行、可校验的证据	<code>skill_validation_report_</code> 记录 <code>install</code> 、 <code>dis-</code> <code>cover</code> 、 <code>dry-run</code> 、 <code>con-</code> <code>tract check</code> 、 <code>release</code> <code>check</code>	只写“验证通过”，没有 命令和结果
<code>not_available</code>	诚实标记不可用依赖	前序周资产没真实落 地时，Week09 报告写 <code>not_available</code> ，不 <code>fake</code> <code>passed</code>	为了让交付看起来完整 而伪造通过